

Chapter 21

Validators

Input validation is one of the most important web development tasks that Spring handles extremely well. There are two ways you can validate input in Spring MVC, by using Spring's own validation framework or by utilizing a JSR 303 implementation. This chapter covers both methods of input validation.

Validation Overview

Converters and formatters work on the field level. In an MVC application, they convert or format a **String** to another Java type, such as a **java.util.Date**. A validator, on the other hand, works on the object level. It determines whether or not all fields in an object are valid and some rules are followed.

The sequence of events in an application that employs both formatters and validators goes like this. During the invocation of a controller, one or more formatter will try to convert input strings to field values in the domain object. Once formatting is successful, the validators step in.

For example, an **Order** object may have a **shippingDate** property (of type **Date**, obviously) whose value cannot be earlier than today's date. When the **OrderController** is invoked, a **DateFormatter** would convert a string to a **Date** and assign it to the **shippingDate** property of the **Order** object. If the conversion failed, the user will be redirected to the previous form. If the conversion was successful, a validator will be invoked to check if **shippingDate** is earlier than today.

Now, you may ask if it is wise to move the validation logic to the **DateFormatter**. After all, it is not hard to compare dates. The answer is no. First, the **DateFormatter** may be used for formatting other strings to dates, such as **birthDate** or **purchaseDate**. Both dates have different rules than **shippingDate**. In fact, a date of birth for an employee, for example, cannot be a future date. Second, a validator may work by inspecting the relationship between two or more fields, each of which is supported by a different formatter. For example, given an **Employee** object with a **birthDate** property and a **startDate** property, a validator may rule that there is no way someone starts working in the company before he or she was born. Therefore, a valid **Employee** object must have its **birthDate** property value that is earlier than its **startDate** value. That's the job of a validator.

Spring Validators

Right from the start, Spring has been designed with input validation in mind. This was even before JSR 303 (Java validation specification) was conceived. As such, the Spring Validation framework was commonplace even today, although JSR 303 validators are generally recommended for new projects.

To create a Spring validator, implement the **org.springframework.validation.Validator** interface. This interface is given in Listing 21.1 and has two methods, **supports** and **validate**.

Listing 21.1: The Spring Validator interface

```
package org.springframework.validation;
public interface Validator {
    boolean supports(Class<?> clazz);
    void validate(Object target, Errors errors);
}
```

The **supports** method returns **true** if the validator can handle the specified **Class**. The **validate** method validates the target object and populate the **Errors** object with validation errors.

An **Errors** object is an instance of the **org.springframework.validation.Errors** interface. An **Errors** object contains a list of **FieldError** and **ObjectError** objects. A **FieldError** represents an error that is related to one of the properties in the validated object. For example, if the **price** property of a product must not be negative and the price of a **Product** object being validated is negative, then a **FieldError** needs to be created. An **ObjectError** is any error that is not a **FieldError**. For example, if a **Book** that is for sale in Europe is being purchased on an American online branch, then an **ObjectError** should be raised.

When writing a validator, you don't need to create an error object directly as instantiating **ObjectError** or **FieldError** takes a lot of programming effort. This is because the **ObjectError** class's constructor expects four arguments and the **FieldError** class's constructor takes seven, as you can see in the constructor signatures below.

```
ObjectError(String objectName, String[] codes, Object[] arguments,
            String defaultMessage)
```

```
FieldError(String objectName, String field, Object rejectedValue,
            boolean bindingFailure, String[] codes, Object[] arguments,
            String defaultMessage)
```

The easier way to add an error to the **Errors** object is by calling one of the **reject** or **rejectValue** methods on the **Errors** object. You call **reject** to add an **ObjectError** and **rejectValue** to add a **FieldError**.

Here are some of the method overloads of **reject** and **rejectValue**.

```
void reject(String errorCode)
```

```
void reject(String errorCode, String defaultMessage)
```

```
void rejectValue(String field, String errorCode)
void rejectValue(String field, String errorCode,
    String defaultMessage)
```

Most of the time, you just need to pass an error code to the **reject** or **rejectValue** method. Spring can then look up the error code against a properties file to obtain the corresponding error message. You can also pass a default message that will be used if the error code is not found.

The error messages in an **Errors** object can be displayed on an HTML page by using the **errors** tag of the form tag library. Error messages can be localized through the internationalization feature that Spring supports. More about internationalization can be found in Chapter 22, “Internationalization.”

The ValidationUtils Class

The **org.springframework.validation.ValidationUtils** class is a utility that can help you write a Spring validator. Instead of writing this.

```
if (firstName == null || firstName.isEmpty()) {
    errors.rejectValue("price");
}
```

you can use the **ValidationUtils** class’s **rejectIfEmpty** method like this.

```
ValidationUtils.rejectIfEmpty("price");
```

Or instead of this.

```
if (firstName == null || firstName.trim().isEmpty()) {
    errors.rejectValue("price");
}
```

you can write this.

```
ValidationUtils.rejectIfEmptyOrWhitespace("price");
```

Here are the complete method overloads of **rejectIfEmpty** and **rejectIfEmptyOrWhitespace** methods in **ValidationUtils**.

```
public static void rejectIfEmpty(Errors errors, String field,
    String errorCode)
public static void rejectIfEmpty(Errors errors, String field,
    String errorCode, Object[] errorArgs)
public static void rejectIfEmpty(Errors errors, String field,
    String errorCode, Object[] errorArgs, String defaultMessage)
public static void rejectIfEmpty(Errors errors, String field,
    String errorCode, String defaultMessage)
public static void rejectIfEmptyOrWhitespace(Errors errors,
    String field, String errorCode)
```

```

public static void rejectIfEmptyOrWhitespace(Errors errors,
    String field, String errorCode, Object[] errorArgs)

public static void rejectIfEmptyOrWhitespace(Errors errors,
    String field, String errorCode, Object[] errorArgs,
    String defaultMessage)

public static void rejectIfEmptyOrWhitespace(Errors errors,
    String field, String errorCode, String defaultMessage)

```

In addition, **ValidationUtils** also has an **invokeValidator** method for invoking a validator.

```

public static void invokeValidator(Validator validator,
    Object obj, Errors errors)

```

You learn how to use this useful tool in the example in the next section.

A Spring Validator Example

The **app21a** application contains a validator named **ProductValidator** for validating **Product** objects. The **Product** class for **app21a** is given in Listing 21.2 and the **ProductValidator** class in Listing 21.3.

Listing 21.2: The Product class

```

package app21a.domain;
import java.io.Serializable;
import java.util.Date;

public class Product implements Serializable {
    private static final long serialVersionUID = 748392348L;
    private String name;
    private String description;
    private Float price;
    private Date productionDate;

    // getters and setters not shown
}

```

Listing 21.3: The ProductValidator class

```

package app21a.validator;

import java.util.Date;
import org.springframework.validation.Errors;
import org.springframework.validation.ValidationUtils;
import org.springframework.validation.Validator;
import app21a.domain.Product;

public class ProductValidator implements Validator {

    @Override
    public boolean supports(Class<?> klass) {

```

```

        return Product.class.isAssignableFrom(klass);
    }

    @Override
    public void validate(Object target, Errors errors) {
        Product product = (Product) target;
        ValidationUtils.rejectIfEmpty(errors, "name",
            "productname.required");
        ValidationUtils.rejectIfEmpty(errors, "price",
            "price.required");
        ValidationUtils.rejectIfEmpty(errors, "productionDate",
            "productiondate.required");
        Float price = product.getPrice();
        if (price != null && price < 0) {
            errors.rejectValue("price", "price.negative");
        }
        Date productionDate = product.getProductionDate();
        if (productionDate != null) {
            // The hour, minute, second components of productionDate
            // are 0
            if (productionDate.after(new Date())) {
                System.out.println("salah lagi");
                errors.rejectValue("productionDate",
                    "productiondate.invalid");
            }
        }
    }
}

```

The **ProductValidator** validator is a very simple validator. Its **validate** method checks if a **Product** has a name and a price and the price is not negative. It also makes sure the production date is not later than today.

The Resource File

You don't need to explicitly register a validator. However, if you want the error messages to be taken from a properties file, you need to tell Spring where to find the file by declaring a **messageSource** bean. Here is the messageSource bean declaration in **app21a**.

```

<bean id="messageSource" class="org.springframework.context.support.
➡ReloadableResourceBundleMessageSource">
    <property name="basename" value="/WEB-INF/resource/messages"/>
</bean>

```

The bean essentially says that error codes and error messages can be found in the **messages.properties** file under **/WEB-INF/resource**.

Listing 21.4 shows the content of the **messages.properties** file.

Listing 21.4: The messages.properties file

```

productname.required=product.name=Please enter a product name
price.required=Please enter a price
productiondate.required=Please enter a production date

```

```
productiondate.invalid=Invalid production date. Please ensure the
production date is not later than today.
```

The Controller class

You can use a Spring validator in a controller class by directly instantiating the validator class. The **saveProduct** method in the **ProductController** class in Listing 21.5 creates a **ProductValidator** and calls its **validate** method. To check if the validator generated an error message, call the **hasErrors** method on the **BindingResult**.

Listing 21.5: The ProductController class

```
package app21a.controller;

import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;

import app21a.domain.Product;
import app21a.validator.ProductValidator;

@Controller
public class ProductController {

    private static final Log logger = LogFactory
        .getLog(ProductController.class);

    @RequestMapping(value = "/product_input")
    public String inputProduct(Model model) {
        model.addAttribute("product", new Product());
        return "ProductForm";
    }

    @RequestMapping(value = "/product_save")
    public String saveProduct(@ModelAttribute Product product,
        BindingResult bindingResult, Model model) {

        ProductValidator productValidator = new ProductValidator();
        productValidator.validate(product, bindingResult);

        if (bindingResult.hasErrors()) {
            FieldError fieldError = bindingResult.getFieldError();
            logger.info("Code:" + fieldError.getCode() + ", field:"
                + fieldError.getField());

            return "ProductForm";
        }
    }
}
```

```

        // save product here

        model.addAttribute("product", product);
        return "ProductDetails";
    }
}

```

Another way of using a Spring validator is by writing an **initBinder** method in your controller, and passing the validator to the **WebDataBinder** and calling its **validate** method.

```

@org.springframework.web.bind.annotation.InitBinder
public void initBinder(WebDataBinder binder) {
    // this will apply the validator to all request-handling methods
    binder.setValidator(new ProductValidator());
    binder.validate();
}

```

Passing a validator to the **WebDataBinder** will apply the validator to all request-handling methods in the controller class.

Alternatively, you can annotate the object argument to be validated with **@javax.validation.Valid**. For example,

```

public String saveProduct(@ModelAttribute Product product,
    BindingResult bindingResult, Model model) {

```

The **Valid** annotation type is defined in JSR 303 and I will defer any discussion of JSR 303 until the next section.

Testing the Validator

To test the validator in **app21a**, direct your browser to this URL.

```
http://localhost:8080/app21a/product_input
```

You will see a blank Product form. If you click the Add Product button without entering any value, you will be redirected back to the Product form and this time there will be error messages from the validator, as shown in Figure 21.1.

The screenshot shows a web browser window with the title 'Add Product Form'. The address bar shows 'localhost:8080/app21a/product_save'. The main content area displays a form titled 'Add a product'. The form has four input fields: 'Product Name', 'Description', 'Price', and 'Production Date'. Above the 'Product Name' field is a red error message 'Please enter a product name'. Above the 'Price' field is a red error message 'Please enter a price'. Above the 'Production Date' field is a red error message 'Please enter a production date'. The 'Price' field is preceded by an asterisk '*'. At the bottom right of the form are two buttons: 'Reset' and 'Add Product'.

Figure 21.1: The ProductValidator in action

JSR 303 Validation JSR 303, “Bean Validation” (published in November 2009) and JSR 349, “Bean Validation 1.1” (published in May 2013) specify a set of API’s for applying constraints on object properties through annotations. JSR 303 and JSR 349 can be downloaded from these URLs, respectively.

<http://jcp.org/en/jsr/detail?id=303>

<http://jcp.org/en/jsr/detail?id=349>

Of course, a JSR is just a specification document and of little use until someone writes an implementation of it. In the case of bean validation JSR’s, there are currently two implementations. The first is Hibernate Validator, which is currently at version 5 and implement both JSR 303 and JSR 349. It can be downloaded from this site.

<http://sourceforge.net/projects/hibernate/files/hibernate-validator/>

The second implementation is Apache BVal, which implements JSR 303 only and can be downloaded from this site.

<http://bval.apache.org/downloads.html>

Apache BVal 0.5 was the latest version at the time of writing and did not seem stable enough. For this apparent reason, We are using Hibernate Validator for the accompanying application (**app21b**).

As a side note, the following is an important website for everything related to Java bean validation.

<http://beanvalidation.org>

With JSR 303, there is no validator to write. Instead, you embed constraints in the domain class by using JSR 303 annotation types. The list of constraints is given in Table 21.1

Constraint	Description	Example
@AssertFalse	Applied to a boolean property. The value of the property must be false.	@AssertFalse boolean hasChildren;
@AssertTrue	Applied to a boolean property. The value of the property must be true.	@AssertTrue boolean isEmpty;
@DecimalMax	The value of the property must be a decimal value lower than or equal to the specified value.	@DecimalMax("1.1") BigDecimal price;
@DecimalMin	The value of the property must be a decimal value greater than or equal to the specified value.	@DecimalMin("0.04") BigDecimal price;
@Digits	The value of the property must be within the specified range. The integer attribute specifies the maximum integral digits for the number and the fraction attribute the maximum fractional digits for the number.	@Digits(integer=5, fraction=2) BigDecimal price;
@Future	The value of the property must be a date in the future.	@Future Date shippingDate;
@Max	The value of the property must be an integer lower than or equal to the specified value	@Max(150) int age;
@Min	The value of the property must be an integer greater than or equal to the specified value	@Min(0) int age;
@NotNull	The property must not be null	@NotNull String firstName;
@Null	The property must be null	@Null String testString;
@Past	The value of the property must be a date in the past.	@Past Date birthDate;
@Pattern	The value of the property must match the specified regular expression	@Pattern(regexp="\\d{3}") String areaCode;
@Size	The size of the property must be within the specified range.	Size(min=2, max=140) String description;

Table 21.1: JSR 303 Constraints

Once you understand how it works, JSR 303 validation is easier to use than Spring validators. As with Spring validators, you can overwrite error messages from a JSR 303 validator by using property keys in this format in your properties file.

constraint.object.property

For example, to overwrite a message from **@Size** constraining the **name** property of a **Product** object, use this key in your properties file.

```
Size.product.name
```

To overwrite a message from **@Past** constraining the **productionDate** property of a **Product** object, use this key.

```
Past.product.productionDate
```

A JSR 303 Validator Example

The **app21b** application shows JSR 303 input validation in action. The application is a modified version of **app21a** with a few differences. First, there is no **ProductValidator** class. Second, the jar files from the Hibernate Validator library have been added to **WEB-INF/lib**.

Listing 21.6 shows the **Product** class whose **name** and **productionDate** fields have been annotated with JSR 303 annotation types.

Listing 21.6: The Product class in app21b

```
package app21b.domain;
import java.io.Serializable;
import java.util.Date;

import javax.validation.constraints.Past;
import javax.validation.constraints.Size;

public class Product implements Serializable {
    private static final long serialVersionUID = 78L;

    @Size(min=1, max=10)
    private String name;

    private String description;
    private Float price;

    @Past
    private Date productionDate;

    // getters and setters not shown
}
```

In the **ProductController** class's **saveProduct** method, you must annotate the **Product** argument with **@Valid**, as shown in Listing 21.7.

Listing 21.7: The ProductController class in app21b

```
package app21b.controller;

import javax.validation.Valid;
import org.apache.commons.logging.Log;
```

```

import org.apache.commons.logging.LogFactory;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.RequestMapping;
import app21b.domain.Product;

@Controller
public class ProductController {

    private static final Log logger = LogFactory
        .getLog(ProductController.class);

    @RequestMapping(value = "/product_input")
    public String inputProduct(Model model) {
        model.addAttribute("product", new Product());
        return "ProductForm";
    }

    @RequestMapping(value = "/product_save")
    public String saveProduct(@Valid @ModelAttribute Product product,
        BindingResult bindingResult, Model model) {

        if (bindingResult.hasErrors()) {
            FieldError fieldError = bindingResult.getFieldError();
            logger.info("Code:" + fieldError.getCode() + ", object:"
                + fieldError.getObjectName() + ", field:"
                + fieldError.getField());
            return "ProductForm";
        }

        // save product here

        model.addAttribute("product", product);
        return "ProductDetails";
    }
}

```

To customize error messages from the validator, two keys are used in the `messages.properties` file. This file is printed in Listing 21.8.

Listing 21.8: The `messages.properties` file in `app21b`

```

Size.product.name = Product name must be 1 to 10 characters long
Past.product.productionDate=Production date must a past date

```

To test the validator in **app21b**, direct your browser to this URL.

`http://localhost:8080/app21b/product_input`

Summary

In this chapter you learned the two types of validators you can use in your Spring MVC applications, Spring MVC validators and JSR 303 validators. JSR 303 validators are the recommended validation method for new projects as JSR 303 is a formal Java specification.